# System przetwarzania danych oparty na pojedynczym oraz współpracujących modelach LLM

Oliwier Komorowski Jakub Nowak

7 listopada 2026

# Streszczenie

Projekt inżynierski dotyczy współpracy wielu dużych modeli językowych (LLM) w rozwiązywaniu złożonych problemów.

**Słowa kluczowe:** duży model językowy (LLM), współpraca wielu modeli, umysł rojowy, Ollama
**Dziedzina nauki i techniki, zgodnie z wymogami OECD:** Nauki o komputerach i informatyka

# Abstract

This engineering project concerns the cooperation of multiple large language models (LLMs) in solving complex problems.

**Keywords:** large language model (LLM), multi-model collaboration, hive mind, Ollama
**Field of science and technology in accordance with OECD requirements:** Computer Science and Information technology

# Spis treści


**1 Wstęp i cel pracy** **4**
- 1.1 Motywacja . . . 4
  - 1.1.1 Dynamiczny rozwój dziedziny . . . 4
  - 1.1.2 Nowe możliwości . . . 5
- 1.2 Cel pracy . . . 6
- 1.3 Struktura pracy . . . 6

**2 Wprowadzenie do dziedziny** **8**
- 2.1 Generatywna sztuczna inteligencja . . . 8
  - 2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji . . . 8
  - 2.1.2 Zasada działania i zastosowania . . . 8
  - 2.1.3 Ograniczenia generatywnej sztucznej inteligencji . . . 9
- 2.2 Duże modele językowe . . . 10
  - 2.2.1 Definicja i architektura . . . 10
  - 2.2.2 Uczenie dużych modeli językowych . . . 10
  - 2.2.3 Sterowanie, wnioskowanie i wyzwania wdrożeniowe . . . 11

**3 Technologie i narzędzia** **13**
- 3.1 Wybrane języki programowania . . . 13
- 3.2 Biblioteki . . . 13
- 3.3 Wybrane modele językowe . . . 14
- 3.4 Narzędzia . . . 16
- 3.5 Wymagania sprzętowe . . . 16

**4 Projekt systemu** **18**
- 4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM . . . 18
- 4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM 18

**5 Badania i analiza** **19**
- 5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi . . . 19
  - 5.1.1 Benchmarki . . . 19
  - 5.1.2 Metryki oceny . . . 19
- 5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu . . . 19

# Rozdział 1

# Wstęp i cel pracy

Aktualnie duże model językowe (ang. LLM, Large Language Model) wykorzystywane są w różnych obszarach obszarach życia. Wysokie zainteresowanie sztuczną inteligencją sprzyja rozwojowi tej dziedziny. Skutkuje to pojawianiem się nowych możliwości wykorzystania technologii przetwarzania języka naturalnego oraz zrewolucjonizowaniem sposobu, w jakim systemy informatyczne przetwarzają i analizują dane nieustrukturyzowane.

W miarę jak rosną oczekiwania wobec systemów opartych na LLM, pojedynczy model może okazać się niewystarczający. Typowymi ograniczeniami są kontekst, koszt obliczeniowy pojedynczego zapytania czy podatność na błędy wynikające z braku weryfikacji własnych odpowiedzi. Naturalną odpowiedzią na te ograniczenia jest koncepcja architektur wielomodelowych, w których kilka instancji LLM współpracuje ze sobą.

## 1.1 Motywacja

Motywacją do realizacji projektu jest rosnące zainteresowanie sztuczną inteligencją, a w szczególności dużymi modelami językowymi oraz wykorzystaniem ich możliwości. Ciekawym zagadnieniem jest zbadanie wpływu współpracy wielu modeli na jakość odpowiedzi.

### 1.1.1 Dynamiczny rozwój dziedziny

W ciągu ostatnich dekad można było zauważyć dynamiczny postęp w obszarze sztucznej inteligencji (SI, ang. Artificial Intelligence, AI). Rozwój tej dziedziny doprowadził do znacznego zwiększenia liczby prowadzonych na całym świecie badań, co znajduje odzwierciedlenie między innymi w rosnącej liczbie cytowań publikacji poświęconych tej tematyce. Z przeprowadzonych analiz bibliometrycznych wynika, że w latach 2012-2022 liczba publikacji dotyczących sztucznej inteligencji oraz Big Data rosła wykładniczo, przy czym szczególnie wyraźne przyspieszenie tego trendu nastąpiło od 2019 roku [1]. Rozwiązania oparte na SI znalazły zastosowanie w wielu różnych obszarach, między innymi w medycynie, edukacji, biznesie i zarządzaniu, usługach rządowych (inteligentne miasta), finansach (prognozowanie i analiza), a także w turystyce oraz sektorze rozrywkowym.

Jedną z dziedzin rozwijających się w ramach sztucznej inteligencji jest przetwarzanie języka naturalnego (ang. Natural Language Processing, NLP). Jej głównym zadaniem jest umożliwienie komputerom pracy z językiem używanym przez człowieka poprzez jego odpowiednie przekształcanie i interpretowanie. Pozwala to na automatyczną analizę, generowanie oraz przetwarzanie treści tekstowych. Istotnym elementem rozwoju NLP stały się również duże modele

językowe (LLM), których możliwości w ciągu ostatnich lat znacząco wzrosły. Modele te ewoluowały z pasywnych systemów przetwarzania tekstu w aktywne podmioty (agenty AI) zdolne do autonomicznego planowania, rozumowania i wchodzenia w interakcje ze środowiskiem [2]. Ich popularność gwałtownie zwiększyła się w listopadzie 2022 roku, kiedy firma OpenAI udostępniła model ChatGPT-3.5. Narzędzie to w bardzo krótkim czasie zdobyło ogromną liczbę użytkowników - w nieco ponad dwa miesiące przekroczyło próg stu milionów użytkowników.

Coraz częściej duże modele językowe służą również do wyszukiwania iformacji i rozwiązywania problemów. Przykładowo generatywna sztuczna inteligencja (GenAI) może pomóc rozwiązać zadanie matematyczne, wytłuamczyć zagadnienie naukowe, czy udzielić odpowiedzi na pytanie o zdrowie. W zależności od zapytania i sposobu jego sformułowania model dopasowywuje sposób odpowiedzi. Zamiast samego wyniku potrafi przedstawić sobób w jaki do niego doszedł lub dostosować wypowiedź do wymaganego poziomu. Ze względu na to ważne jest sformułowanie odpowiedniego promptu. Zbyt ogólen polecenie czy brak kontekstu może skończyć się odpowiedzią niepełną, błędną albo różną od oczekiwanej. Z halucynacjami próbuje się z kolei walczyć za pomocą wielowarstwowych zabezpieczeń, monitorowania przebiegu pracy (Workflow Monitoring) i odpowiedzi w ustrukturyzowanym formacie (JSON), co pozwala lepiej skalibrować pewność modelu [1]. Ma to duże znaczenie tam, gdzie liczy się dokładność, czyli choćby w matematyce czy medycynie. Odpowiedzi modelu należy tam traktować jako wsparcie, a nie jako w pełni wiarygodne źródło.

### 1.1.2 Nowe możliwości

Dynamiczny rozwój sztucznej inteligencji oraz metod przetwarzania języka naturalnego przyczynił się do powstania nowych sposobów wykorzystania komputerów w codziennych zadaniach. Szczególne znaczenie mają tutaj generatywna sztuczna inteligencja oraz duże modele językowe, które ewoluowały z pasywnych systemów odpowiedzi w autonomiczne agenty AI. Umożliwiają one nie tylko uzyskiwanie informacji, ale również samodzielne korzystanie z zewnętrznych narzędzi, planowanie wieloetapowych zadań i interakcję ze środowiskiem w celu realizacji złożonych celów [3]. Rozwiązania te mogą być wykorzystywane między innymi do wspomagania nauki, rozwiązywania zadań matematycznych, wyjaśniania zagadnień z różnych dziedzin czy udzielania odpowiedzi na pytania wymagające analizy dostarczonych informacji.

#### Duże modele językowe

Duże modele językowe (ang. Large Language Models, LLM) w ostatnich latach stały się jednym z najważniejszych obszarów rozwoju sztucznej inteligencji. Jeszcze stosunkowo niedawno bezpośrednia komunikacja z komputerem za pomocą języka naturalnego wymagała specjalistycznych rozwiązań, natomiast obecnie użytkownik może formułować pytania i polecenia w sposób zbliżony do rozmowy z drugą osobą. Dzięki temu modele językowe mogą pełnić funkcję narzędzia wspomagającego, a obecnie coraz częściej tworzą złożone systemy wieloagentowe (ang. Large Language Model-based Multi-Agent Systems, MASs) [2], [3]. W takich strukturach wyspecjalizowane jednostki (np. agent-programista, agent-tester) współpracują ze sobą, co pozwala na naturalny podział pracy, większą odporność na błędy oraz rozwiązywanie problemów o stopniu złożoności przekraczającym możliwości pojedynczego modelu. Przykładowe zastosowania obejmują generowanie i analizowanie tekstu, tłumaczenie treści, tworzenie podsumowań, odpowiadanie na pytania, a także pomoc w rozwiązywaniu problemów matematycznych i logicznych.

Istotną zaletą części dostępnych modeli językowych jest ich otwarty charakter (ang. open-source software), który umożliwia wykorzystanie ich we własnych aplikacjach. Platformy takie

jak Hugging Face udostępniają szeroki wybór modeli, które można dostosować do określonych zastosowań i zintegrować z tworzonym oprogramowaniem. Pozwala to między innymi na budowanie systemów wykorzystujących modele językowe do analizy pytań użytkownika oraz generowania odpowiedzi bez konieczności tworzenia własnego modelu od podstaw.

Wykorzystanie modeli uruchamianych lokalnie daje również możliwość ograniczenia konieczności przesyłania danych do zewnętrznych usług. Przy odpowiednio wydajnej infrastrukturze model może zostać uruchomiony bezpośrednio na komputerze użytkownika, a komunikacja z nim może odbywać się za pośrednictwem lokalnego interfejsu API. Takie rozwiązanie może mieć szczególne znaczenie w przypadku aplikacji przetwarzających informacje, dla których istotne są prywatność i bezpieczeństwo. Lokalna obsługa modelu pozwala bowiem zachować dane w obrębie własnej infrastruktury, ograniczając ryzyko ich nieautoryzowanego udostępnienia.

## 1.2 Cel pracy

Celem pracy jest stworzenie systemu przetwarzania, który umożliwia efektywną współpracę dużych modeli językowych w celu rozwiązywania złożonych problemów. Projekt zakłada implementację takiego systemu. Głównym zamysłem pracy jest zbadanie wpływu współpracy wielu modeli na jakość odpowiedzi oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji takich rozwiązań.

Aplikacja zostanie zaimplementowana w języku Python i będzie umożliwiała komunikację z lokalnie uruchomionymi dużymi modelami językowymi. Połączenie z modelami zostanie zrealizowane za pośrednictwem interfejsu API platformy Ollama, dzięki czemu przetwarzanie zapytań będzie odbywało się w całości na urządzeniu użytkownika, bez konieczności korzystania z zewnętrznych usług chmurowych. Modele biorące udział w działaniu systemu będą współdzieliły jedną wspólną przestrzeń, służącą do wymiany spostrzeżeń oraz wyników przeprowadzanych obliczeń. Dane wyjściowe poszczególnych modeli będą łączone w jeden ciąg znaków, który następnie stanie się kontekstem dostępnym dla wszystkich uczestników. Dzięki temu każdy z modeli będzie mógł odwoływać się do wniosków sformułowanych przez pozostałe, co pozwoli na wspólne rozwiązywanie problemów i wzajemne uzupełnianie się w trakcie pracy.

Interakcja z aplikacją będzie odbywała się z poziomu terminala za pomocą tekstowego interfejsu wiersza poleceń (ang. Command Line Interface, CLI). Aplikacja będzie umożliwiać podgląd aktualnego stanu przetwarzania za pomocą logów generowanych w trakcie działania systemu i wyświetlania ich użytkownikowi na bieżąco. Odpowiedź końcowa również zostanie zaprezentowana w terminalu. Główny nacisk zostanie położony na funkcjonalność i logikę działania usługi, a nie na rozbudowaną warstwę prezentacji.

Projekt ma na celu zbadanie efektywności współpracy wielu dużych modeli językowych w rozwiązywaniu złożonych problemów oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji systemów opartych na współpracujących LLM.

## 1.3 Struktura pracy

Praca liczy sześć głównych rozdziałów, ułożonych tak, aby krok po kroku poznawać zagadnienie oraz kolejne etapy realizacji projektu. W rozdziale pierwszym przedstawiono tematykę pracy i określono jej cel. Rozdział drugi ma charakter teoretyczny i wprowadza w omawianą dziedzinę, wyjaśniając pojęcie generatywnej sztucznej inteligencji oraz dużych modeli

językowych. W rozdziale trzecim opisano technologie użyte w projekcie, do których należą Python, Ollama i modele językowe. Rozdział czwarty poświęcono projektowi systemu, w którym szczegółowo omówiono poszczególne komponenty. W rozdziale piątym zaprezentowano wyniki badań dotyczące skuteczności współpracy wielu modeli, wydajności modeli językowych oraz jakości generowanych przez nie odpowiedzi. Rozdział ostatni zawiera podsumowanie uzyskanych rezultatów oraz kierunki dalszego rozwoju pracy.

W tabeli 1.1 przedstawiono podział odpowiedzialności autorów za poszczególne rozdziały i podrozdziały.

Tabela 1.1: Podział odpowiedzialności autorów za poszczególne rozdziały

| **Autor** | **Numery rozdziałów i podrozdziałów** |
|---|---|
| Olwier Komorowski | 1, 3.1, 3.4 |
| Jakub Nowak | 2, 3.2, 3.3, 3.5, 4, 5, 6 |

# Rozdział 2

# Wprowadzenie do dziedziny

Niniejszy rozdział wprowadza w dziedzinę, w której osadzona jest praca. Najpierw przedstawia generatywną sztuczną inteligencję: jej definicję, zasadę działania, zastosowania oraz ograniczenia. Następnie omawia duże modele językowe, czyli klasę modeli generatywnych, na której opiera się system MLLMS. Wprowadzone pojęcia są podstawą dla kolejnych rozdziałów, które opisują technologie, projekt systemu i wyniki badań.

## 2.1 Generatywna sztuczna inteligencja

### 2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji

Sztuczna inteligencja (SI) obejmuje metody, które pozwalają komputerom wykonywać zadania wymagające zwykle ludzkiej inteligencji. Jej istotną częścią jest uczenie maszynowe (ang. Machine Learning, ML), w którym zależności nie są programowane wprost, lecz wyznaczane przez algorytm na podstawie danych. Szczególnym przypadkiem uczenia maszynowego są sztuczne sieci neuronowe (ang. Artificial Neural Networks), czyli modele zbudowane z warstw prostych jednostek obliczeniowych, których parametry dobierane są w procesie uczenia. Sieci składające się z wielu warstw określa się mianem uczenia głębokiego (ang. Deep Learning, DL). Umożliwiło ono modelowanie bardzo złożonych zależności w danych, takich jak tekst, obraz czy dźwięk.

Na uczeniu głębokim opiera się współczesna generatywna sztuczna inteligencja (ang. Generative Artificial Intelligence, GenAI). Według [4] są to techniki obliczeniowe zdolne do generowania pozornie nowych, sensownych treści, takich jak tekst, obrazy czy dźwięk, na podstawie danych treningowych. Modele generatywne nie ograniczają się więc do klasyfikowania danych wejściowych lub przewidywania pojedynczej wartości, lecz tworzą nowe dane, przypominające te, na których były uczone. Wśród nich szczególne miejsce zajmują duże modele językowe, opisane w sekcji 2.2.3.

### 2.1.2 Zasada działania i zastosowania

Model generatywny uczy się, jak wyglądają dane, na których był trenowany. Uczy się rozkładu prawdopodobieństwa tych danych, oznaczanego $p(x)$, lub rozkładu $p(x \mid c)$, jeśli wynik zależy od kontekstu $c$, na przykład od polecenia użytkownika (ang. prompt). Oznacza to, że model ocenia, jak prawdopodobny jest każdy możliwy kolejny token (fragment tekstu) przy danym zapytaniu i dotychczas napisanym tekście. Odpowiedź powstaje token po tokenie.

To, który token zostanie wybrany, zależy od sposobu dekodowania. Przy dekodowaniu zachłannym (ang. greedy decoding) model zawsze wybiera token najbardziej prawdopodobny. Dla tego samego zapytania daje więc w praktyce zawsze tę samą odpowiedź. Przy próbkowaniu (ang. sampling) token jest losowany, przy czym tokeny bardziej prawdopodobne mają większą szansę na wybór. Siłę tej losowości ustawia się parametrem zwanym temperaturą (ang. temperature): im niższa temperatura, tym częściej wybierany jest najbardziej prawdopodobny token. Przy próbkowaniu kolejne odpowiedzi mogą się różnić. Żeby je powtórzyć, ustala się ziarno generatora liczb losowych (ang. seed).

Model dyskryminatywny działa inaczej. Nie uczy się, jak wyglądają dane, tylko jak przypisać dane wejściowe $x$ do etykiety $y$. Formalnie uczy się więc wprost rozkładu $p(y \mid x)$ [5]. Przykładem jest filtr spamu, który dla każdej wiadomości ocenia, czy jest niechciana.

Oba rodzaje modeli przypisują prawdopodobieństwa, ale czemu innemu. Model dyskryminatywny przypisuje je etykietom, a jego wynikiem jest zwykle etykieta o największym prawdopodobieństwie. Model generatywny, przypisuje je kolejnym tokenom, a jego wynikiem jest tekst złożony z wybranych tokenów. Widać to na pytaniu wielokrotnego wyboru. Model dyskryminatywny przypisałby prawdopodobieństwo każdej z odpowiedzi A, B, C i D i wskazał najbardziej prawdopodobną. Model generatywny pisze tekst, w którym podaje literę odpowiedzi.

Treści tworzone przez modele generatywne mogą mieć różną postać. Według [4] są to między innymi tekst, obrazy i wideo, mowa i muzyka oraz kod źródłowy. Niniejsza praca dotyczy wyłącznie tekstu.

Generatywna sztuczna inteligencja jest wykorzystywana w edukacji, nauce, medycynie, biznesie i codziennej pracy z informacją, między innymi do odpowiadania na pytania, streszczania i tłumaczenia tekstów, wyjaśniania zagadnień oraz pomocy w programowaniu. Szczególnie istotne dla tej pracy są zadania wymagające rozumowania i wiedzy specjalistycznej, takie jak rozwiązywanie zadań matematycznych i odpowiadanie na pytania medyczne. W tych dziedzinach wiarygodność odpowiedzi ma duże znaczenie, co prowadzi do ograniczeń omówionych w następnej podsekcji.

### 2.1.3 Ograniczenia generatywnej sztucznej inteligencji

Mimo szerokich możliwości generatywna sztuczna inteligencja ma ograniczenia, które wynikają bezpośrednio ze sposobu jej działania.

Najbardziej znanym z nich są halucynacje (ang. hallucinations), czyli generowanie treści brzmiących wiarygodnie, lecz nieprawdziwych. Wyróżnia się wśród nich halucynacje dotyczące faktów, gdy treść jest sprzeczna z weryfikowalną wiedzą o świecie, oraz halucynacje dotyczące wierności, gdy odpowiedź odbiega od polecenia lub kontekstu [6]. Ponieważ model losuje tokeny zgodnie z wyuczonym rozkładem i nie sprawdza prawdziwości treści, odpowiedź błędna wygląda tak samo jak poprawna, a przekonujący ton utrudnia wykrycie błędu. Ma to duże znaczenie w dziedzinach takich jak medycyna, w których błędna informacja może prowadzić do szkody.

Z tym wiąże się ograniczona zdolność samoweryfikacji. Klasyczny model językowy nie sprawdza własnej odpowiedzi, zanim ją poda. Zmieniły to modele rozumujące (ang. reasoning models), które przed udzieleniem odpowiedzi generują tok rozumowania i w jego trakcie sprawdzają swoje kroki pośrednie. Takie sprawdzenie nie jest jednak w pełni obiektywne i nie gwarantuje poprawności. Zwykle potwierdza jedynie wcześniejszy wynik, a rzadko wykrywa i poprawia błąd [7]. Pojedynczy model pozostaje więc jednocześnie autorem i jedynym recenzentem odpowiedzi.

Kolejnym ograniczeniem jest wrażliwość na sformułowanie polecenia. Słaby prompt, czyli ogólnikowy lub pozbawiony kontekstu, zwykle daje odpowiedź niepełną lub błędną, natomiast dobrze zbudowany, precyzujący zadanie i oczekiwany format, poprawia wynik. Nawet drobne zmiany formy polecenia, niezmieniające jego sensu, mogą zmienić odpowiedź.

Ograniczenia te, w szczególności ograniczona samoweryfikacja, stanowią punkt wyjścia dla koncepcji współpracy wielu modeli, opisanej w kolejnych rozdziałach.

## 2.2 Duże modele językowe

### 2.2.1 Definicja i architektura

Duże modele językowe to nic innego jak sieci neuronowe o bardzo dużej liczbie parametrów, trenowane na ogromnych zbiorach tekstów. Opierają się one na architekturze Transformer [8], która umożliwia efektywne modelowanie zależności w długich sekwencjach. Dzięki temu mogą one rozpoznawać i interpretować złożone teksty. Przed ich pojawieniem się, do tego typu zadań stosowano rekurencyjne sieci neuronowe (RNN [9]), które zaczęły mieć trudności wraz z zwiększeniem modeli i długości sekwencji. Przez potrzebę wykonywania sekwencyjnych obliczeń były one mało efektywne.

Pojawienie się obecnych modeli uznaje się za wyraźny przełom w przetwarzaniu języka naturalnego (NLP) i w całej dziedzinie sztucznej inteligencji (SI)[3]. Odpowiedzialna za to jest wcześniej wspomniana architektura Transformer. Opublikowana w 2017 roku praca ”Attention Is All You Need”[8] wprowadziła mechanizm samo-uwagi (ang. self-attention), który pozwala modelom dynamicznie ważyć znaczenie poszczególnych słów w kontekście całej sekwencji. Poza tym architektura ta umożliwia równoległe przetwarzanie danych, co znacząco zwiększa efektywność treningu i wnioskowania w porównaniu do wcześniejszych podejść opartych na RNN. Od tego momentu zaczęto tworzyć coraz to większe modele. Liczba parametrów współczesnych modeli wynosi miliardy, a w największych przypadkach sięga bilionów ($10^{12}$). Przełomem okazały się rodziny modeli takie jak GPT, LLaMA i Gemini, które pokazały, że wzrost rozmiaru, ilości danych i mocy obliczeniowej przekłada się na jakość działania [3].

Skala ta przynosi też zjawisko określane jako zdolności emergentne (ang. emergent abilities). Po przekroczeniu pewnego progu liczby parametrów i mocy obliczeniowej model zaczyna wykonywać zadania, do których nie był wprost trenowany. Przykładem może być wnioskowanie przez analogię czy uczenie zero-shot, czyli rozwiązywanie zadania bez przykładów w poleceniu [3]. Poniżej tego progu takich umiejętności nie obserwuje się. Właśnie ta cecha, a nie sam rozmiar, odróżnia duże modele językowe od wcześniejszych modeli statystycznych i neuronowych.

### 2.2.2 Uczenie dużych modeli językowych

Modele bazowe uczą się na tekstach pozbawionych jakiejkolwiek struktury, a ich zadanie jest jedno: odgadnąć kolejne słowo, znając to, co padło wcześniej. Taki trening wystarcza, by model nauczył się budować zdania poprawne pod względem składni. Matematycznie opisuje się to jako prawdopodobieństwo warunkowe. W każdym kroku model wybiera słowo, które w danym kontekście jest najbardziej prawdopodobne:

$$w_t = \arg\max_w P(w \mid w_1, w_2, \ldots, w_{t-1})$$

Jakość przewidywań mierzy funkcja straty, którą w trakcie uczenia minimalizujemy:

$$L = -\sum_{t=1}^{T} \log P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

W efekcie sieć przyswaja zasady gramatyki i ogólną budowę języka. Na tak przygotowanym modelu wykonuje się potem dostrajanie pod konkretne zastosowania. Jednym z nich jest wykonywanie poleceń wydawanych przez użytkownika. Po odpowiednim sformułowaniu prompta taki model potrafi przetłumaczyć tekst, odpowiedzieć w narzuconym formacie albo przyjąć wskazaną rolę. Dwie ostatnie umiejętności wykorzystano w tej pracy.

Dane treningowe pochodzą przeważnie z internetu i zostały napisane przez ludzi. Stąd bierze się przypuszczenie, że modele przejmują również część wzorców, według których ludzie podejmują decyzje.

### 2.2.3 Sterowanie, wnioskowanie i wyzwania wdrożeniowe

#### Inżynieria promptów

Zachowanie dużego modelu językowego można w znacznej mierze ukierunkować bez zmiany jego wag. Zajmuje się tym inżynieria promptów (ang. prompt engineering): dobór instrukcji, kontekstu, formatu odpowiedzi i przykładów tak, by model dał wynik zgodny z oczekiwaniami. Kosztuje to znacznie mniej niż ponowne trenowanie lub dostrajanie modelu, bo wymaga tylko zmiany tekstu wejściowego. Dlatego prompting jest zwykle pierwszym narzędziem, po które sięga się przy dopasowywaniu modelu do konkretnego zadania.

#### Chain-of-Thought

Chain-of-Thought (CoT) to technika, która zamiast prosić o samą odpowiedź, skłania się model do rozbicia złożonego problemu na logiczne kroki pośrednie i dopiero na ich podstawie sformułowania wyniku [2]. Dzięki rozłożeniu zadania na mniejsze kroki zwiększa się czytelność i staranność rozumowania modelu. Model przestaje ”zgadywać”końcowy wynik. Dzięki temu łatwiej radzi sobię z problemami wymagającymi kilku następujących po sobie operacji. Technika ta świetnie sprawdza się w arytmetyce, logice czy planowaniu. Jako efekt uboczny uzyskujemy też jawny zapis toku rozumowania, który można przejrzeć i ocenić.

Samo CoT ma jednak słabość: zwykle łączy się je z dekodowaniem zachłannym (ang. greedy decoding), w którym w każdym kroku wybierany jest najbardziej prawdopodobny token. Powstaje wtedy jedna ścieżka rozumowania i jeden błąd w jej środku przesądza o wyniku. Metoda Self-Consistency (CoT-SC) zastępuje ten mechanizm wielokrotnym próbkowaniem: model generuje wiele różnych ścieżek rozumowania, zamiast zatrzymać się na jednej, zachłannej[10][11]. Następnie wybierana jest odpowiedź najbardziej spójna, czyli taka, do której prowadzi najwięcej niezależnych ścieżek

Cenę za tę poprawę płaci się w obliczeniach. Każde dodatkowe próbkowanie to kolejne wywołanie modelu, więc koszt i czas rosną proporcjonalnie do liczby generowanych ścieżek. W zastosowaniach lokalnych, gdzie zasoby sprzętowe są ograniczone, ma to realne znaczenie przy wyborze liczby próbek.

#### Halucynacje

Mimo imponujących możliwości duże modele językowe mają ograniczenia. Najczęściej wskazuje się wśród nich halucynacje, czyli generowanie błędnych lub niemających oparcia w

faktach treści [3]. Problem nie polega wyłącznie na tym, że model się myli. Błędne informacje podaje on tym samym płynnym, zdecydowanym tonem co poprawne, więc z samej formy odpowiedzi trudno ocenić, czy można jej zaufać. Wynika to ze sposobu działania modelu: wybiera on tekst najbardziej prawdopodobny w danym kontekście, a nie sprawdza go z wiarygodnym źródłem wiedzy.

Ryzyko rośnie w systemach złożonych z kilku współpracujących modeli lub agentów. Halucynacja jednego z nich może wywołać efekt kaskadowy: kolejne elementy systemu przyjmują błędny wynik jako pewnik i budują na nim dalsze rozumowanie, przez co błąd się powiela i wzmacnia [6]. Im dłuższy łańcuch przetwarzania, tym trudniej wskazać, w którym miejscu pojawił się pierwotny fałsz.

# Rozdział 3

# Technologie i narzędzia

Istotny był wybór odpowiednich technologii i narzędzi. Ważne było aby system był wydajny i jego poszczególne komponenty łatwe do wdrożenia dzięki odpowiednim udokumentowaniu. Celem tego rozdziału jest przedstawienie użytych w projekcie technologii oraz uzasadnienie ich wyboru.

## 3.1 Wybrane języki programowania

Do realizacji projektu zdecydowano się na język programowania Python, który dzięki swojej prostocie, szybkości implementacji oraz sporej liczbie bibliotek, był najlepszym wyborem z pośród innych języków. Dodatkowym atutem jest jego popularność pośród istniejących już rozwiązań uczenia maszynowego. Stanowiło to dodatkową inspirację podczas projektowania systemu.

## 3.2 Biblioteki

W projekcie wykorzystano szereg bibliotek, które skróciły czas implementacji, ograniczając potrzebę pisania kodu od podstaw i zapewniając sprawdzone rozwiązania typowych problemów programistycznych. Obok modułów biblioteki standardowej Pythona (json, random, re, subprocess) użyto bibliotek zewnętrznych, wymienionych w pliku requirements.txt:

**requests**

Biblioteka do wysyłania zapytań HTTP, uznawana w ekosystemie Pythona za standardowe narzędzie do tego celu. Udostępnia prosty interfejs do wysyłania żądań oraz odbierania i przetwarzania odpowiedzi, w tym danych w formacie JSON. W projekcie służy do komunikacji z interfejsem API platformy Ollama: klasa agenta wysyła zapytanie do punktu końcowego czatu, a następnie odbiera wygenerowaną odpowiedź modelu. Obsłużono przy tym wyjątki zgłaszane przez bibliotekę, dzięki czemu problem z połączeniem nie przerywa działania całego systemu.

**huggingface_hub**

Oficjalna biblioteka kliencka do współpracy z repozytorium Hugging Face Hub, w którym publikowane są modele i zbiory danych wykorzystywane w uczeniu maszynowym. Umożliwia pobieranie plików oraz zarządzanie lokalną pamięcią podręczną, dzięki czemu raz pobrane

dane nie muszą być ściągane ponownie. W projekcie nie jest wywoływana bezpośrednio — korzysta z niej biblioteka datasets, pobierając za jej pośrednictwem zbiory danych z repozytorium.

**datasets**

Biblioteka rozwijana przez Hugging Face, przeznaczona do ładowania i przetwarzania zbiorów danych. Zapewnia jednolity interfejs dostępu do danych niezależnie od ich pierwotnego formatu oraz zajmuje się ich pobraniem i lokalnym buforowaniem.

**pandas**

Biblioteka do analizy i przetwarzania danych tabelarycznych, której podstawową strukturą jest obiekt DataFrame. Pozwala wygodnie filtrować, grupować i agregować dane, a także sterować sposobem ich prezentacji. W projekcie wykorzystano ją na etapie analizy wyników: zapisane wyniki eksperymentów są wczytywane do obiektu DataFrame, a następnie zestawiane i podsumowywane, co umożliwia porównanie skuteczności poszczególnych konfiguracji systemu.

**pyarrow**

Biblioteka będąca implementacją Apache Arrow w języku Python. Apache Arrow definiuje kolumnowy format reprezentacji danych w pamięci, który umożliwia ich wydajne przetwarzanie i odczyt. W projekcie nie jest wywoływana bezpośrednio — pełni rolę silnika odczytującego pliki w formacie Parquet, w którym udostępniane są zbiory danych z repozytorium Hugging Face. Z tego silnika korzystają biblioteki datasets oraz pandas.

**fastparquet**

Kolejna biblioteka do odczytu i zapisu plików w formacie Parquet, czyli kolumnowym formacie przechowywania danych, który dobrze je kompresuje i umożliwia szybki dostęp do wybranych kolumn. Pełni tę samą funkcję co pyarrow i stanowi dla niej alternatywę, dzięki czemu pliki Parquet można wczytać niezależnie od tego, który z silników jest dostępny w środowisku. Podobnie jak pyarrow nie jest wywoływana w kodzie wprost, lecz wykorzystywana przez biblioteki wyższego poziomu.

**sympy**

Biblioteka do obliczeń symbolicznych, która pozwala przetwarzać wyrażenia matematyczne, przekształcać je i porównywać ze sobą. W projekcie służy do sprawdzania równoważności odpowiedzi matematycznych, ponieważ ta sama wartość może zostać zapisana na wiele sposobów, np. jako ułamek zwykły lub dziesiętny. Wykorzystano ją w module oceniającym w dwóch miejscach: przy porównywaniu odpowiedzi modelu z odpowiedzią wzorcową oraz przy ustalaniu, czy modele biorące udział w debacie doszły do zgodnego rozwiązania.

## 3.3 Wybrane modele językowe

Dobór modeli podyktowany był założeniami projektu. System opiera się na współpracy dwóch modeli. Jej wpływ na jakość odpowiedzi podlega badaniu, dlatego potrzebne były modele

pochodzące od różnych twórców, a nie dwie instancje tego samego modelu. Ponadto modele powinny z przyzwoitą dokładnością rozwiązywać zadania z obu dziedzin, na których oparto badania, czyli matematyki i medycyny. Kolejnym wymogiem było, aby były to modele o otwartych wagach (ang. open-weight), dostępne w bibliotece Ollamy i możliwe do uruchomienia lokalnie. Ostatnim kryterium było ograniczenie sprzętowe – oba modele musiały mieścić się w pamięci operacyjnej komputera, na którym prowadzono badania (szczegóły opisano w sekcji 3.5). W ramach tego ograniczenia wybierano modele o możliwie największej liczbie parametrów, ponieważ jest ona jednym z czynników wpływających na jakość generowanych odpowiedzi.

Tabela 3.1 zestawia modele rozważane na etapie wyboru. Wytłuszczono modele użyte w projekcie.

Tabela 3.1: Modele rozważane w projekcie

| **Model** | **Twórca** | **Parametry** | **Decyzja i powód** |
|---|---|---|---|
| **gemma4:12b** | **Google** | **12 mld** | **Wybrany.** Najnowsza generacja rodziny Gemma, dobra jakość na zadaniach matematycznych, mieści się w limicie pamięci. |
| **qwen3.5:9b** | **Alibaba** | **9 mld** | **Wybrany.** Najnowsza generacja rodziny Qwen, dobra jakość na zadaniach matematycznych, inna rodzina niż Gemma. |
| llama3.1:8b | Meta | 8 mld | Odrzucony. Wyraźnie słabsza jakość odpowiedzi. |
| qwen2.5:7b | Alibaba | 7 mld | Odrzucony. Jakość przeciętna oraz starsza generacja względem Qwen3.5. |
| gemma2:9b | Google | 9 mld | Odrzucony. Jakość przeciętna oraz starsza generacja względem Gemma 4. |
| gemma2:2b | Google | 2 mld | Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi. |
| phi4-mini | Microsoft | 3,8 mld | Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi. |
| deepseek-r1:14b | DeepSeek-AI | 14 mld | Odrzucony. Bardzo dobre wyniki w matematyce, zbliżone do wybranych modeli, ale wyższe zużycie pamięci. |

Wybrane zostały modele gemma4:12b (Google) oraz qwen3.5:9b (Alibaba). Pochodzą od różnych twórców i należą do różnych rodzin, co pozwala na zróżnicowane odpowiedzi i rozumowanie. Mają przy tym zbliżony rozmiar, dzięki czemu ewentualne różnice w wynikach nie wynikają wyłącznie z liczby parametrów. Oba reprezentowały najnowszą generację swoich rodzin w momencie wyboru.

Wybór opierał się na wstępnych próbach, a nie na pełnym porównaniu. Każdy z rozważanych modeli uruchomiono w aplikacji w taki sam sposób jak w badaniach opisanych w rozdziale 5, na zadaniach z obu rozpatrzanych dziedzin. Odpowiedzi porównywano z odpowiedziami wzorcowymi i sprawdzano, czy model zachowuje wymagany format odpowiedzi. Próby te miały charakter orientacyjny i służyły jedynie odrzuceniu modeli wyraźnie słabszych, dlatego ich wyniki nie zostały zapisane.

## 3.4 Narzędzia

### Git

Git to narzędzie służące do zarządzania wersjami projektu, umożliwiające monitorowanie wprowadzanych zmian w kodzie oraz równoczesną pracę wielu osób nad tym samym projektem.

W projekcie pełnił funkcję podstawowego systemu kontroli wersji. Każdy członek zespołu pracował na oddzielnej gałęzi (ang. branch), a po zakończeniu prac wprowadzone zmiany były łączone z główną gałęzią projektu (ang. main branch). Takie podejście ułatwiało współpracę, ograniczało ryzyko występowania konfliktów oraz pozwalało na zachowanie uporządkowanej, liniowej historii zmian.

### GitHub

GitHub to platforma wykorzystująca system Git, która pozwala na przechowywanie oraz zarządzanie repozytoriami znajdującymi się na zdalnych serwerach. Oprócz podstawowych funkcji oferuje również możliwość wzajemnego przeglądania i weryfikowania kodu przez członków zespołu, a także automatycznego uruchamiania procesów sprawdzających poprawność projektu, takich jak wykonywanie testów czy weryfikacja kompilacji kodu w ramach CI (ang. Continuous Integration). Funkcja przeglądu kodu pozwalała zespołowi na bieżąco kontrolować jakość wprowadzanych zmian, wykrywać potencjalne błędy oraz utrzymywać kod w odpowiednim stanie.

### Ollama API

Ollama to oprogramowanie umożliwiające lokalne uruchamianie oraz zarządzanie wybranymi modelami językowymi typu LLM (ang. Large Language Model).

W projekcie zostało wykorzystane do obsługi modeli działających bezpośrednio na lokalnej infrastrukturze, co pozwalało na korzystanie z ich możliwości bez konieczności przesyłania danych do zewnętrznych usług. Komunikacja pomiędzy aplikacją a uruchomionymi modelami odbywała się za pośrednictwem interfejsu API, umożliwiając przesyłanie zapytań oraz odbieranie wygenerowanych odpowiedzi.

## 3.5 Wymagania sprzętowe

Badania prowadzono na komputerze MacBook Pro z procesorem Apple M4 Pro i 24 GB pamięci operacyjnej. Pamięć ta jest zunifikowana (współdzielona przez procesor i układ graficzny). Model uruchomiony na GPU zajmuje zatem te same 24 GB, z których korzystają system operacyjny i pozostałe programy.

Model językowy musi w całości mieścić się w pamięci na czas generowania odpowiedzi. Główną część tej pamięci zajmują wagi, czyli parametry modelu. Oprócz wag miejsce zajmuje także pamięć podręczna kluczy i wartości (ang. key-value cache, KV cache). Aby wygenerować kolejny token, model musi uwzględnić wszystkie wcześniejsze tokeny. Wyniki obliczeń dla nich są zapisywane w KV cache, dzięki czemu nie trzeba ich liczyć od nowa przy każdym kolejnym wykonaniu. Im dłuższy kontekst, tym więcej pamięci zajmuje ten cache [12]. W niekwantyzowanej wersji bf16 (16-bitowej), zajęta pamięć wyniosłaby 24 GB dla modelu gemma4:12b [13] i 19 GB dla qwen3.5:9b [14]. Pierwszy z nich zająłby więc całą pamięć komputera. Problem rozwiązuje kwantyzacja, czyli zapis wag z mniejszą liczbą bitów kosztem niewielkiej straty dokładności. Oba użyte modele mają kwantyzację Q4_K_M (kwantyzacja czterobitowa). Tabela 3.2 zestawia parametry modeli i zajmowane przez nie zasoby.

Tabela 3.2: Zasoby zajmowane przez użyte modele

| **Model** | **Parametry** | **Kwantyzacja** | **Dysk** | **Pamięć w działaniu** | **Procesor** |
|---|---|---|---|---|---|
| gemma4:12b | 11,9 mld | Q4_K_M | 7,6 GB | 8,1 GB | 100% GPU |
| qwen3.5:9b | 9,7 mld | Q4_K_M | 6,6 GB | 5,5 GB | 100% GPU |

Obie wartości zajętej pamięci pochodzą z polecenia `ollama ps`.

Ollama domyślnie utrzymuje model w pamięci przez 5 minut od ostatniego użycia [12] i może mieć załadowanych kilka modeli jednocześnie, jeśli mieszczą się w dostępnej pamięci [12]. W systemie modele biorące udział w debacie są wywoływane kolejno, a polecenie `ollama stop` wykonywane jest po zakończeniu całego eksperymentu. Podczas debaty obu modeli możliwa jest zatem obecność obu w pamięci, a ich łączne zapotrzebowanie, równe 13,6 GB, mieści się w dostępnych 24 GB.

# Rozdział 4

# Projekt systemu

## 4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM

## 4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM

# Rozdział 5

# Badania i analiza

## 5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi

### 5.1.1 Benchmarki

### 5.1.2 Metryki oceny

## 5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu

# Spis tabel

1.1 Podział odpowiedzialności autorów za poszczególne rozdziały . . . . . 7

3.1 Modele rozważane w projekcie . . . . . 15
3.2 Zasoby zajmowane przez użyte modele . . . . . 17

# Spis rysunków

# Bibliografia


[1] Y. Cheng, Y. Xu, C. Yu, Y. Zhao, "HAWK: A Hierarchical Workflow Framework for Multi-Agent Collaboration," *arXiv preprint arXiv:2507.04067*, 2025. URL: https://arxiv.org/abs/2507.04067

[2] Y. Yang, Q. Peng, J. Wang, Y. Wen, W. Zhang, "LLM-based Multi-Agent Systems: Techniques and Business Perspectives," *arXiv preprint arXiv:2411.14033*, 2024. URL: https://arxiv.org/abs/2411.14033

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2024. URL: https://arxiv.org/abs/2501.06322

[4] S. Feuerriegel, J. Hartmann, C. Janiesch, P. Zschech, "Generative AI," *Business & Information Systems Engineering*, t. 66, nr 1, s. 111–126, 2024. URL: https://arxiv.org/abs/2309.07930

[5] A. Y. Ng, M. I. Jordan, "On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes," w: *Advances in Neural Information Processing Systems 14 (NIPS 2001)*, 2001. URL: https://proceedings.neurips.cc/paper/2001/hash/7b7a53e239400a13bd6be6c91c4f6c4e-Abstract.html

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, t. 43, nr 2, s. 1–55, 2025. URL: https://arxiv.org/abs/2311.05232

[7] Q. Long, K. J. Jiang, J. Chen, X. Guo, L. Gan, W. Wang, "Self-Verification Dilemma: Experience-Driven Suppression of Overused Checking in LLM Reasoning," *arXiv preprint arXiv:2602.03485*, 2026. URL: https://arxiv.org/abs/2602.03485

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, t. 30, 2017. URL: https://arxiv.org/abs/1706.03762

[9] A. Sherstinsky, "Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network," *Physica D: Nonlinear Phenomena*, t. 404, 2020. URL: https://arxiv.org/abs/1808.03314

[10] J. Li, Q. Zhang, Y. Yu, Q. Fu, D. Ye, "More Agents Is All You Need," *Transactions on Machine Learning Research*, 2024. URL: https://arxiv.org/pdf/2402.05120

[11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, D. Zhou, S. Narang, A. Chowdhery, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," w: *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023. URL: https://arxiv.org/pdf/2203.11171

[12] Ollama. “Ollama FAQ,” dostęp 9 października 2026. URL: https://docs.ollama.com/faq

[13] Ollama. “gemma4: tagi modelu w bibliotece Ollamy,” dostęp 9 października 2026. URL: https://ollama.com/library/gemma4/tags

[14] Ollama. “qwen3.5: tagi modelu w bibliotece Ollamy,” dostęp 9 października 2026. URL: https://ollama.com/library/qwen3.5/tags